

Evaluating an Optimized Inverse Data Regression Model for Weakness and Correlation Between Variables

Erick Arias*
SFSU

E. Wes Bethel†
SFSU

ABSTRACT

In the steps approaching the improvement of the efficiency of Falkner Planck Equation solvers (like CQL3D) and its required accompanying ray tracing software (like GENRAY) we must identify the relationships that exist between conditions inside the fusion chamber toroid, the tokamak. Finding these relationships may aid in creating faster predictive models to determine the desired qualities of wave power deposition [power] and current drive [curtor] that exist inside the tokamak. To do so, we will create a random forest regressor model that will take “real life” data, recorded from the current time-costly programs, and will predict a set of considered conditions that could have created such power and curtor respectively—like an inverse model of accompanying research’s ideal model. The inverse model will be optimized before creation and evaluated through its error ranges for each condition variable.

1 INTRODUCTION

In the current environment of fusion research, an interest in the latest fusion reactor concept, the ‘advanced tokamak’ (AT), has lead to the search for an effective manner to develop a model to project the conditions inside the toroidally symmetric plasma. The conditions of the plasma inside are crucial to maintaining fusion and even possibly ignition. What can be used to affect the conditions of the AT’s toroid are auxiliary actuators, which require detailed information on the radio frequency (RF) heating and current drive profiles to maintain magnetohydrodynamic (MHD) stable overall current and pressure profiles [2]. Additionally, current drive is often required at specific radial locations where an actuator may not be providing enough power to maintain ideal conditions. The required information can be calculated using the Fokker–Planck equation (can be done using CQL3D software). However, the process of using that requires values for variables which are priorly obtained through a wave solver algorithm (TORIC+Petra-M or GENRAY) [5]. This process however is very computationally expensive and timely—sometimes taking hours to days to compute. Experimental models to expedite this process and compute the desired qualities of wave power deposition [power] and current profiles [curtor] have been formed through three different methods: random forest regression, gaussian process, and multilayer perceptron. We are provided with “real life” observed data for the power and curtor output for which given deep learning models have been composed to challenge this issue. However, we seek to build on the current progress by supplying accompanying information of the relationships of conditional variables in the AT and their influence on their dependent power and curtor outputs.

Our selected approach of evaluating these relationships is to observe them in an “inverse model” (power/curtor to find conditional variables) of the originally desired model (conditional variables to find power/curtor). We’ll achieve this by training and optimizing a random forest regressor model and later drawing out possible ranges

of individual conditional variables or sets/cases of conditional variables where error is the highest.

2 RELATED WORK

Forward models for the focused inverse model have already been produced by scientists and experts in their respective focus from MIT, UC Berkeley, and Princeton and share a relationship of the purpose of this paper’s work. Their work aimed to create machine learning models use a database of more than 16,000 GENRAY/CQL3D simulations for training, validation and testing [5]. The models had been optimized, validated, and converted to ONNX format for universal acceptance and agreeable performance. In this paper we will use the same data and similar processes to generate an inverse model for which we can explore relationships among the forward model’s input variables (conditional variables) which may have not been noticed and considered when generating a model. The focus of this research is to provide data for correlation between conditional variables and power profiles, curtor profiles, and x-ray data to improve efficiency and accuracy in future prediction models.

The 9 conditional variables given to us from Berkeley physics team have properties relating the plasma in the tokamak. $B0_eqdsk$ refers to magnetic field at center of plasma. A higher $B0_eqdsk$ allows waves to penetrate into higher density. $R0_eqdsk$ refers to the major radius of plasma, where larger plasmas require more power. $Ate0$ refers to the central temperature of plasma, where higher temperature leads to stronger wave damping. $Dense0$ refers to the central density of plasma, where higher density requires more power to drive current. Also higher density requires higher $B0_eqdsk$ to allow waves to propagate. $Elecfld$ refers to the DC toroidal electric field, where higher $elec fld$ drives more Ohmic “background” current. Ip_scale refers to the total plasma current, where the higher ip_scale the higher poloidal B-field which changes how rays travel through plasma. Higher ip_scale also improves plasma confinement. $Zeff$ is effective charge of plasma, where a higher $Zeff$ requires more power to drive current. Lh_npara1 refers to the parallel refractive index of wave, where a lower value for lh_npara1 is more efficient for CD, but can’t penetrate to high density. Lastly, lh_power refers to RF power and wave damping is “quasi-linear” with it. [6]

When considering ways to optimize our inverse model we have several contenders. One such method would be to consider the (k-fold) cross-validation method where the data is split into a subsection, once or several times, and we are able to evaluate the risk of running a case for an algorithm. This method does hold well for assessing risk, or error, as it prevents overfitting. Part of data (the training sample) is used for training each algorithm, and the remaining part (the validation sample) is used for estimating the error of the algorithm. The method has errors in classification problems and draws into possible shifting test data [3] and has little impact on some test data sets. Although in our case for regression we avoid the bulk of pitfalls with this method, making it a perfect candidate for a step in validation. Another approach would be hyperparameter optimization which would include adjustments in the parameters of a model to assess error and, in turn, validate it. Through adjustment of the basis of the algorithm we are to calculate error and in turn optimize our random forest regression inverse model for accuracy [1].

*e-mail:earias1@mail.sfsu.edu

†email:ewbethel@sfsu.edu

3 IMPLEMENTATION

The data points given for validation and which our models seek to mimic inversely are that of GENRAY/CQL3D calculations that take into account condition variables in a fusion reactor, one of which is current drive [curtor], to produce a resultant RF power absorption [power] value.

Normally when “working forward” to make these calculations possible we require other knowledge of the variables that make up the conditions of the plasma torus. We consider a total of 9 zero dimension variables that meet this criteria and will refer to those variables, when creating the sets random forest regressor model, under the names: 'B0_eqdsk', 'R0_eqdsk', 'ate0', 'dense0', 'elecflld', 'lh_npara1', 'lh_power', 'zeff', 'ip_scale'.

However in our inverse models given the data for 9 zero-dimensional variables and their resultant curtor and power, we seek to create inverted random forest regression models to predict the 9 zero-dimensional variables using their relational trend with the resultant curtor and power respectively. However, alongside these inverted random forest regression models we will consider a secondary set of models that will seek to achieve the same goal of finding the models' output 9 zero-dimensional variables but will use an input of power/curtor and x-ray conditions in the toroid. The set of models that also incorporate x-ray data is expected to not have significant impact as it is not directly correlated to power and curtor calculation but may hold some value if a trend is sensed in the toroid conditions. If a trend were to be found then it would be a significant find for forward (not inverted) model refining. To create these sets of models we will optimize the models using k-fold data set validation and hyper-parameterization of the sci-kit library's random forest regression model function. Following their creation we will test the models' prediction error across 1000 sets of normalized data from Genray/CQL3D and display the error of the data across the set range of 9 zero-dimensional variables respectively.

The random forest regression model will be generated, trained, and applied using a pre-built function from the sklearn/scikit-learn library [4]. The function is dependent on a list of preset mutable parameters and the choice for the input data. The data will initially be divided using a test-to-train data split function also provided from scikit-learn for training and testing the created regression model. The goal is to create the regression model and optimize it for accuracy.

The code done in this section will utilize modules such as RandomForestRegressor from sklearn.ensemble for regression model formation, KFold from sklearn.model_selection for data set separation into groups, train_test_split from sklearn.model_selection for data set separation into two groups, mean_squared_error from sklearn.metrics for MSE computation, onnx for ONNX. model manipulation, from skl2onnx.common.data_types to import FloatTensorType to pass model information to a converter, onnxmltools to convert an sklearn model to an ONNX model, numpy for array manipulation, matplotlib.pyplot for graphing, and pandas for data frame manipulation.

The repository for the code and data sets is currently private, only being available to the contributors to this paper at the time of this writing, but the all code and work will be noted in the paper. The Jupyter Notebooks for this section of the paper can be found on this github repository when made public: https://github.com/ariaserick25/Modeling_MSE.git

3.1 Hyperparameter Optimization

When there is a large number of decision trees in the random forest and the training data is relatively small, each decision tree may end up capturing and overfitting to some of the noise in the data, rather than the underlying patterns or relationships. In other words, the decision tree may pick up on random fluctuations in the data that are not actually informative, and incorporate them into its prediction function. This can be problematic because when the random

Number of n_estimators	Power MSE	Pow. W/Xray MSE
100	0.05143	0.05246
150	0.05132	0.05232
200	0.05106	0.05222
250	0.05099	0.05205
300	0.05103	0.05200
350	0.05100	0.05192
400	0.05100	0.05184
450	0.05096	0.05185
500	0.05090	0.05176

Table 1: We have the comparison of the calculated MSEs between the power inverse model and the power inverse model that includes x-rays across the n_estimator test range. The max value of n_estimators is set to 500 which is half of the amount of data points.

Number of n_estimators	Curtor MSE	Cur. W/Xray MSE
100	0.06488	0.04665
150	0.06446	0.04644
200	0.06438	0.04633
250	0.06420	0.04641
300	0.06405	0.04638
350	0.06414	0.04631
400	0.06412	0.04633
450	0.06424	0.04633
500	0.06421	0.04633

Table 2: We have the comparison of the calculated MSEs between the curtor inverse model and the curtor inverse model that includes x-rays across the n_estimator test range. The max value of n_estimators is set to 500 which is half of the amount of data points.

forest combines the predictions of all the decision trees, the noise in each tree's prediction can accumulate and result in a less accurate overall prediction. This is why having a large number of decision trees relative to the size of the training data can potentially lead to overfitting, and why it's important to choose an appropriate value for n_estimators.

While the optional parameters to sklearn's regressor model are fairly large, the most notable to test would be the number of trees in the “forest”, n_estimators. In a random forest regressor an input would be put into a series of decision trees of n_estimator size where each tree will provide a prediction of the output. Lastly the regressor takes the mean of the outputs and returns it to the user. The n_estimators value is preset to 100, however depending on size of the data set provided we may find that 100 is not the best number of trees to evaluate the data. We'll consider that the highest number of n_estimators to be considered will be one in which the data set is split into a series of trees such that each tree only has two children. We'll consider that the MAX will be $n/2$, where n is equal to the number of data points. In this study the data set is of size 1000 so to evaluate the best value for n_estimators we will have its value be tested at values incremented from 100 to 500 in 1/8th of the range length and the corresponding total mean squared error (MSE) will be recorded. We consider that the step size of 1/8th of the range length is an effective option as increments of value 1 (the minimum size increment) will have no notable effect as it may very well be indistinguishable from the natural variance in numerical output or mean squared error of the random forest regression model output. [Listing 1]

After checking the MSE of the outputs, we find that the most efficient value for n_estimators are: 500 for the inverse power model with and without x-ray data [Table 1], 300 for the inverse curtor model, and 350 for the inverse curtor model with x-ray data [Table 2].

```

1 input_train, input_test, mse_train, mse_test=
  train_test_split(inputs, mse, test_size=0.20,
    random_state=10)

3 best_n_estimator= 0
4 best_mse= 1
5 counter=0
6 _max = round(n/2)
7 for i in range(100, _max+ round((_max-100)/8) ,
  round((_max-100)/8)):
8   my_n_estimators = i
9   rf_regressor = RandomForestRegressor(
    n_estimators = my_n_estimators, random_state
    =10)

11  rf_regressor.fit(input_train, mse_train)
12  mse_predict = rf_regressor.predict(input_test)
13  mse_of_rfr = ((mse_predict - mse_test)**2).
    mean()

15  if mse_of_rfr < best_mse:
16      best_n_estimator = i
17      best_mse= mse_of_rfr

```

Listing 1: Testing Algorithm for best n_estimators: iterates over selected range and sets apart the number of n_estimators which has the lowest set MSE.

Fold	Power MSE	Pow. W/Xray MSE
1	0.05107	0.04664
2	0.05260	0.04948
3	0.05572	0.05238
4	0.05341	0.04931
5	0.05562	0.05251

Table 3: Comparison of different fold MSE between the inverse power model and the inverse power and x-ray model. The k-fold fold number is reflective of the shuffled data set divided into 5 groups where one is used for test data in each fold.

3.2 K-fold Validation

In theory, the more relevant data a random forest regression model can have the more accurate it should be. Given that we have a limited set of data points, we seek to optimize the performance of the model by selecting the data set to train the model which results in the lowest error against the test data. The method of k-fold validation is the ideal manner to evaluate the data as a group of equal splits. In our case we will have the regression model follow a 80/20 split where 80 percent of the data (800 data points) will be used for training the regression model and 20 percent (200 data points) will be used to test it. That means we'll have the data be split into 5 folds (sections) for the k-fold and evaluate the model with fold 1-5 as the test data in each respective iteration. In this procedure we'll be searching for the lowest mean squared error from each fold. It may be worthwhile to note that in this test we will be setting the n_estimators to the preferred value in section 3.1 and that for this evaluation and testing the values for the random_state parameter set equal to 10 on both the regressor model and test/train split function parameters to maintain the same conditions on each interaction [Listing 2]

After checking the MSE of the outputs, we find that the best fold is: 1 for the inverse power model with and without x-ray data [Table 3], 2 for the inverse curtor model, and 4 for the inverse curtor model with x-ray data [Table 4].

```

1 kf = KFold(n_splits=5, random_state=10, shuffle=
  True) #kf is a generator that yields index
  values for a fold set of n_splits based on an
  input array (ex. kf(array) <-- to be later
  initialized)
2 my_n_estimators = best_n_estimator
3 fold_mse_arr = []
4 for train_index , test_index in kf.split(outputs):
5   X_train , X_test = inputs[train_index], inputs
    [test_index]
6   Y_train , Y_test = outputs[train_index] ,
    outputs[test_index]

8   rf_regressor = RandomForestRegressor(
    n_estimators = my_n_estimators, random_state
    =10)
9   rf_regressor.fit(X_train, Y_train)
10  outputs_predict = rf_regressor.predict(X_test)

12  fold_mse = mean_squared_error(Y_test,
    outputs_predict)
13  fold_mse_arr.append(fold_mse)

15 avg_mse = sum(fold_mse_arr)/5

17 best_mse = 1
18 index = 0
19 for i in range(len(fold_mse_arr)):
20   if fold_mse_arr[i] < best_mse:
21     best_mse = fold_mse_arr[i]
22     index = i
23 generator = kf.split(outputs)

25 if index > 0:
26   skip, skop = next(generator)
27 train_index, test_index = next(generator)

29 inputs_train , inputs_test = inputs[train_index],
    inputs[test_index]
30 outputs_train , outputs_test = outputs[train_index
    ], outputs[test_index]
31 rf_regressor = RandomForestRegressor(
    n_estimators = my_n_estimators)
32 rf_regressor.fit(inputs_train, outputs_train)

```

Listing 2: and sets apart the fold which has the lowest set MSE.]Testing Algorithm for best fold: iterates over k number of fold [5] and sets apart the fold which has the lowest set MSE.

Fold	Curtor MSE	Cur. W/Xray MSE
1	0.06413	0.05190
2	0.06140	0.05343
3	0.06197	0.05257
4	0.06508	0.05171
5	0.06498	0.05548

Table 4: Comparison of different fold MSE between the inverse power model and the inverse power and x-ray model. The k-fold fold number is reflective of the shuffled data set divided into 5 groups where one is used for test data in each fold.

4 EVALUATION

The goal of this section is to display the error of the model sets' prediction of the 9 conditional variables across their respective ranges and to compare the overall mean squared prediction error from the data sets' 9 conditional variables. The data will be drawn from the recently optimized and trained models.

4.1 Computational platform and Software Environment

The work for these experiments were done on an HP Pavilion Laptop - 15z-eh100 with a AMD Ryzen™ 7 5700U (up to 4.3 GHz, 8 MB L3 cache, 8 cores, 16 thread) and a 16 GB DDR4-3200 SDRAM (2 x 8 GB). All coding and compiling was done using Jupyter Notebooks on NERSC CORI with no notable errors or impact on the observed results.

4.2 Methodology

The factors considered to impact the sought conditions of the AT, power and curtor, are inputs in the given set of random forest regression models. The relation between these inputs to our random forest regression models is unknown to us who take the given model as a black box. So, in light of this we seek to identify any relationship between the inputs and the corresponding error of prediction in the random forest regression models to see if any observable variable holds the largest impact on error (MSE).

The first set of graphs will display error difference calculated from the respective random forest regression model across all given test data (200 samples). The outputs set will be the x-axis for the graph; the output set will have 9 individual outputs: 'B0_eqdsk', 'R0_eqdsk', 'ate0', 'dense0', 'elecfd', 'lh_npara1', 'lh_power', 'zeff', 'ip_scale'. The y-axis will be the respective model's output error. In calculating normalized error difference we hope to observe any weakness of the prediction within a smaller range of the normalized range for a given conditional variable.

The second set of graphs evaluated will display our models' overall output prediction mse vs. the individual outputs, which are the conditional variables. The x-axis will have the same outputs set as the previous set of graphs and the y-axis is mse value for the respective conditional variable. We hope to use these set of graphs of the set of models to compare the impact of x-ray data as an input on the respective random forest regressor model (power/curtor). Possibly yield a relationship between the x-ray data and a conditional variable.

The last set of graphs will explore the best and worst cases for the xray excluded model to solidify any information or relationship hypothesized in the previous exploration of graphs.

4.3 Graph Set 1: Inverse Model Error Across Input Variables

This is a set of graphs dedicated to the error difference of the predictions compared to ground truth. It's important to note that the data values have been normalized across the conditional variables' respective ranges— that is to say that the values have been scaled, relative to their ranges in the given data set, from 0 [lowest value in range] to 1 [highest value in range]. Since the data has been normalized, we consider the absolute value of the difference from the predicted to truth to be sufficient in describing the error of the model as compared to percent error (which may develop an unhelpful/un-clear value when dealing with incredibly small values for truth and considerable error). Calculation of error is always in reference to a value and we've found error differences of the normalized values to give the best information given the small values in the truth data set. [Figure 1][Figure 2] As an additional note, there was no significant difference between the x-ray data inclusive and non-inclusive or data that couldn't be derived from only one set of inverse power and curtor models, so there is only one two sets shown.

From the graph set we notice a trend on nearly all conditional variable error graphs for both curtor and power models: there are higher error on the extremes of the normalized conditional variable value range. In the more extreme cases we develop an error trend resembling a function $y = |-x + 0.5| + c$ (where c is some constant). The reason for this trend, is due to prediction range of a random forest regression model. A random forest regression model will not extrapolate, its prediction is the result of a series of decision trees

based off the model train data set. In other words, the predictions of a random forest regression model are limited to range based on the range of the test data. As a result, we can see that an input equal to half its given range will have the highest probability of having the lowest error since the regression model will only have a section of predictions that are at worst a range/2 distance away. The likelihood for error should increase as we head towards either extreme of prediction range. Best reflecting this is our measure of error difference where if a test variable is valued at 0.5, then we have at worst an error difference of 0.5— and this worst case value only increases, approaching 1, as we approach the extremes of the range. Although, it's important to note that this is only true for a poor model or variable predictions. If a model is trained well, then we can notice that the error "lies beneath the $y = |-x + 0.5| + c$ ceiling". That is to say that the likelihood of error is overshadowed by the greater understanding of the trends between the input and output (as seen the data sets' graphs that stray from that y function).

Given the nature of the error of individual inputs in random forest regression models, we can focus on the more y-function distanced graphs as those will like be the ones with lowest overall error. From this trend of the graphs we expect the curtor models to predict ate0, elecfd, and ip_scale best and the power models to predict ate0, lh_power, and ip_scale best. To verify this we will look at the graphs of the following section since they explore the overall mse of the individual conditional variables.

4.4 Graph Set 2: Total MSE Across Conditional Variables

The set of graphs detail the mse across the 9 conditional variables in the different model sets. One set of compared graphs are from models predicting curtor: one only having curtor as an input and the other having curtor and the corresponding x-ray values. The second set compares models that predict power: one only having power as an input and the other having power and the corresponding x-ray values. [Figure 3]

Through individual conditional variables in the overall mse graph sets we can see that curtor models predict ate0, elecfd, and ip_scale best and the power models predict ate0, lh_power, and ip_scale best thus supporting our notion of correlation between the "Ceiling of a $y = |-x + 0.5| + c$ function" and a set of data points that stray from it.

Additionally we can notice the impact of using x-ray data in inverse power and curtor models. We saw that in the optimization process the models which include x-ray data alongside curtor/power had lower overall mse than their respective noninclusive models. However, that "overall" angle of perception is a broad stroke and leaves out some finer details. As seen in this sections graph sets, it appears that the inclusion of x-ray data resulted in improved and worsened accuracy in certain variables.

Looking at the curtor models we can note that the inclusion of x-ray data improved R0_eqdsk, dense0, lh_power, zeff, and ip_scale, while negatively affecting B0_eqdsk, ate0, and elecfd. lh_npara1 was hardly affected [~ 0.001 drop in mse] so we'll consider it negligible. The improvement of R0_eqdsk is significant dropping 0.03804 (42.4%) in mse error. The following were notable changes: dense0 dropped 0.01306 (19.2%), lh_power dropped 0.0132 (19.0%), zeff dropped 0.0179 (20.5%), and ip_scale dropped 0.01518 (33.9%). On the other hand the model did worsen on a few variables: B0_eqdsk grew 0.00632 (7.1%), ate0 grew 0.00667 (41.7%), and elecfd grew 0.0012 (66.7%). With the inclusion of x-rays it appears the model was able to find trends between it and R0_eqdsk, dense0, lh_power, zeff, and ip_scale and not with B0_eqdsk, ate0, and elecfd (however, elecfd could be attributed to minor noise from the x-rays). When considering the impacts of the affected variables in the plasma and tokamak, we can note that those whose mse worsened were variables that were directly associated with affecting wave dampening (B0_eqdsk and ate0) and elecfd (which affects the current drive

...

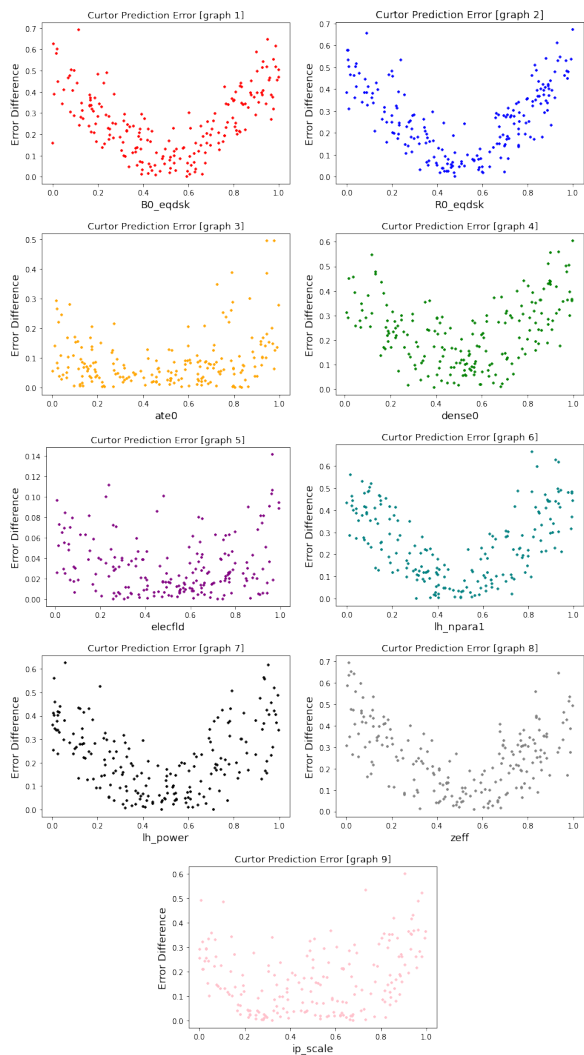


Figure 1: Graph set of error difference across all 9 conditional variables test ranges from inverse curtor model

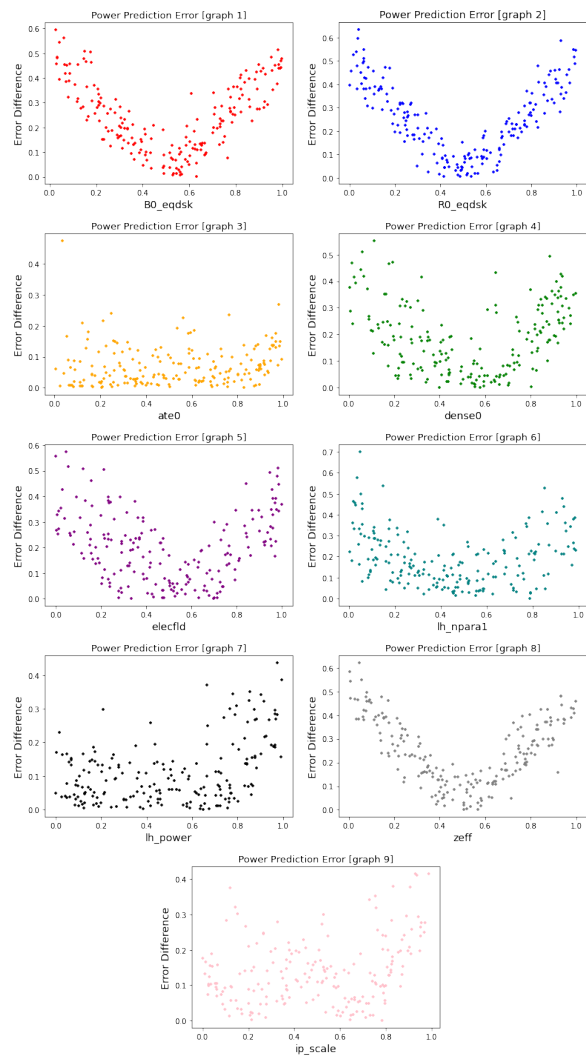


Figure 2: Graph set of error difference across all 9 conditional variables test ranges from inverse power model

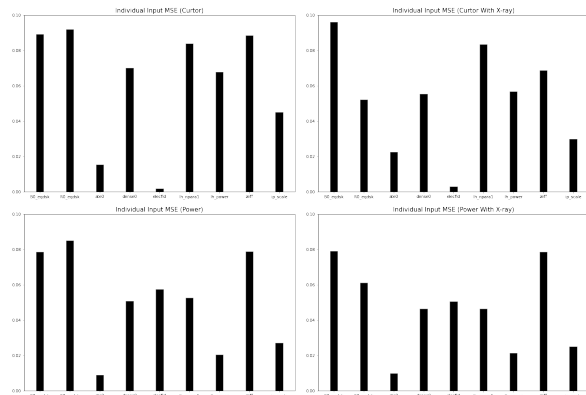


Figure 3: Graph set displaying the mse for all conditional variables in each of the four inverse models

[curtor]).

Shifting to the power models, the inclusion of x-rays has improved R0_eqdsk, dense0, elecfl, lh_npara1, and ip_scale and has negatively affected ate0. Among the positively affect variables: R0_eqdsk's mse value had dropped significantly by 0.03415 (40.2 %), dense0 dropped by 0.0373 (7.5%), elecfl dropped by 0.00632 (11.1%), lh_npara1 dropped by 0.00659 (12.5%), and ip_scale dropped by 0.00265 (9.7%). The only somewhat notably negatively affected variable is ate0, which dropped by 0.00131 (15.4%). The remaining variables, B0_eqdsk, lh_power, and zeff, were hardly affected having a change in mse of less than 0.011. Considering that the only somewhat negatively affected variable is somewhat negligible due to is low change in mse, we can consider that the inclusion of x-ray data in the inverse power model was only helpful. An import note was that, though slightly, the x-ray data had helped the model develop trends with current drive related variables such as dense0, elecfl, lh_npara1, and ip_scale which seemingly could not be drawn well enough from power data alone. Additionally, it appears as though x-ray data may have a considerable correlation to R0_eqdsk since it has significantly improved both curtor and power models' prediction.

4.5 Graph Set 3: Top Four Best Cases of Overall MSE

The work done on the model and the data obtained is stepping stone to an ongoing study that will require the observation of similar model themes but different model structures (multilayer perceptrons, gaussian approximation, etc). An importance to us as researchers is to observe the failings of our model's but also it's strengths and successes. The last set of graphs will look at the best and worst 4 cases of prediction in terms of overall mse. In other words, unlike the last set of graphs where we observed the individual mse of 9 variables in this graph set we focus on the lowest and highest collective mse of all variables. Note that while the x-ray data inclusive models do have a different set for their samples as inclusion has lowered the error for certain variables, this result is trivial to the sort of trend we hope to observe. So, our graph set and observations will only be focused on the non-inclusive data models.

Within the power prediction graph sets we can note a trend that has present among the previous sections of this paper: the closer the values of the conditional variable are to the center of their range, the better the mse— with the exception of trends well understood by regressor model. In the case for the inverse power regression model (regardless of x-ray inclusion) the exceptions to the midpoint trend are ate0, lh_power, and ip_scale and for inverse curtor regression models (regardless of x-ray inclusion) it will be ate0, elecfl, and ip_scale.

When we observe the best case for the inverse power model is one where all variables are close to the midpoint of the range except for ate0. [Figure 4] We can expect this as ate0's trend is well known to the model and does not specifically benefit from being a midpoint value, as shown in Graph Set 1. For the most part, the remaining cases remain close to the midpoint for all but one variable. In the 3rd best case the outlier is zeff and in the fourth it is dense0. There are no such recognized accuracy trends relating to those variables and it shows since the error is still somewhat high among them. Regardless their placement so high in the best error cases observed is due to the fact that the cases where all variables are near their midpoint value is scarce.

Observing the best cases for the curtor inverse model we find similar trends to the best inverse power model but with different exceptions— as shown in the 4th best case with ate0 at an extreme. [Figure 6] However, the 3rd best case is figurative wrench in the wheel that is this noted trend. The 3rd best case has 4 variables moving towards the upper extreme but is still maintain low mse. The trend mentioned in the analysis of Graph Set 2 is simply a reflection of the higher likelihood for error approaching the extremes of a

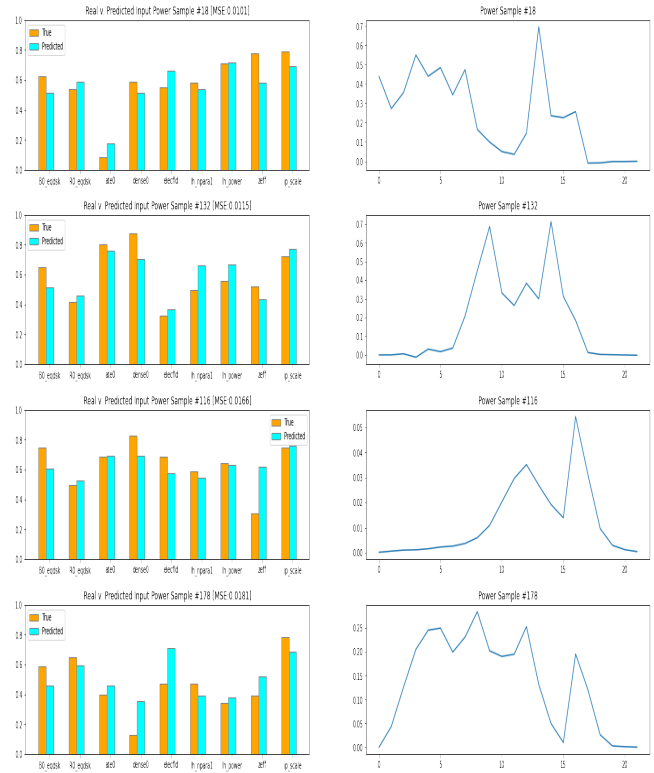


Figure 4: The top 4 best inverse power model prediction cases are shown in descending order [top to bottom]

variables range. So, cases like this can still exist should the noticed trend be true. It would just be unlikely. That is also not to say that there can't be more layers to the extrapolation of the models own set regression trends. Perhaps there is a trend among the respective curtor and power profiles. Work like that would take further physics focused analysis.

Observing the worst cases for the inverse power model and inverse curtor model we can notice that most of the variables are close the extremes. [Figure 5] [Figure 7] As mentioned earlier in Graph Set 2, for random forest regression models with poor prediction power of a variable we can expect the error to increase as we approach the variable's range's extremes. Indeed, the as shown in the prediction bar in all worst cases, the extremes are poorly predicted.

4.6 Findings and Discussion

After optimizing the set of inverse power and inverse curtor models, along with their respective x-ray inclusive counter parts we evaluated their strengths and weakness through evaluation of their predict mse across their conditional variables individually and together as a whole. The error difference across samples for each conditional variable from Graph Set 1 gave the first insight into a trend which shows a fault of the random forest regression models inability to extrapolate. The train data given to a random forest regression model sets the range of output that can be given so we find that test data of value range/2 is least likely to have error compared to values that approach the extremes of the range. However, these trend are mainly notable in variables with poor prediction. As a result we set expectations for the inverse power models to have low mse for ate0, lh_power, and ip_scale and for the inverse curtor models to have low mse for ate0, elecfl, and ip_scale— solely based on the spread of the data points of the graphs (this would be confirmed by Graph Set 2).

After exploring error from both sets of models we find that the

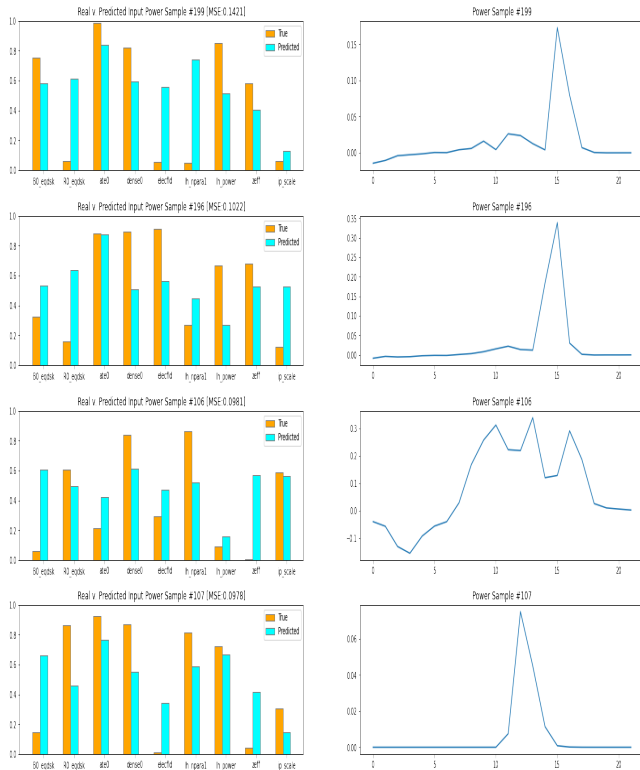


Figure 5: The top 4 worst inverse power model prediction cases are shown in descending order [top to bottom]

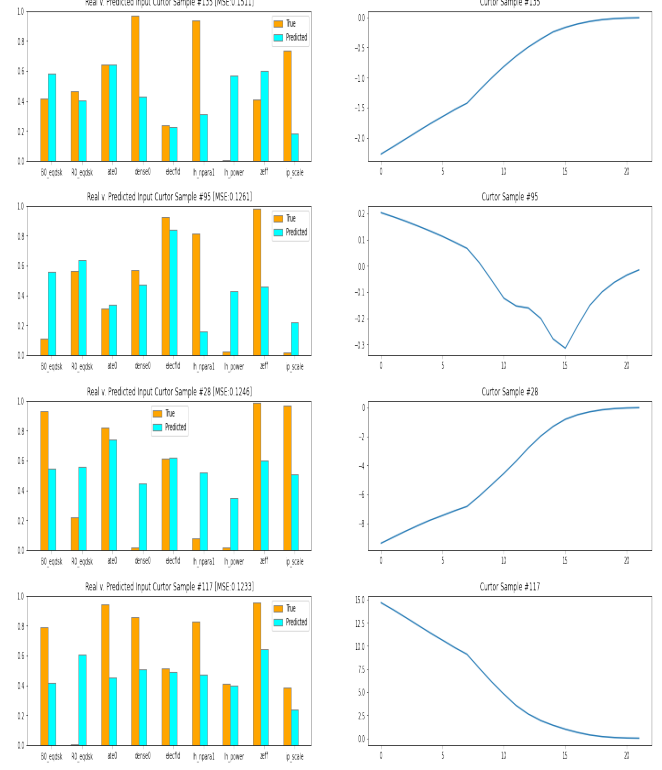


Figure 7: The top 4 worst inverse curtor model prediction cases are shown in descending order [top to bottom]

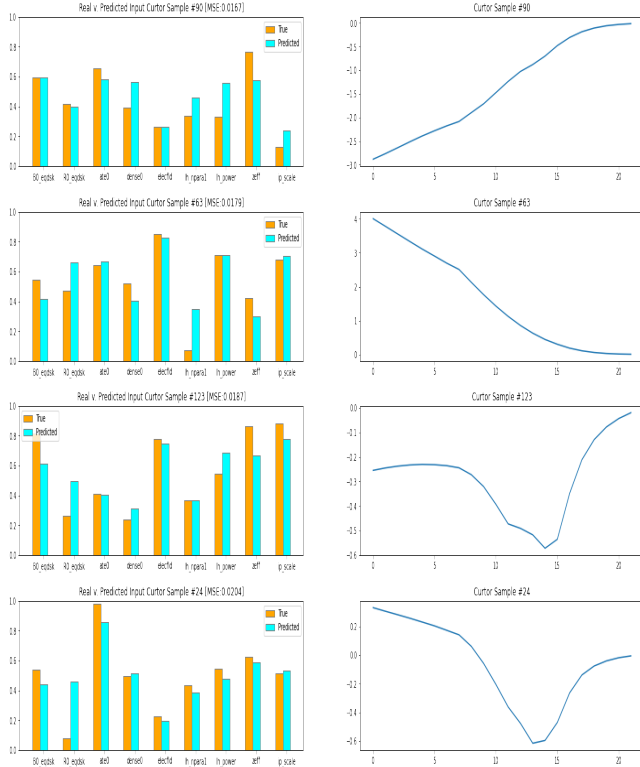


Figure 6: The top 4 best inverse curtor model prediction cases are shown in descending order [top to bottom]

models that include x-ray have a better overall mse and have considerably better prediction for R0_eqdsk— exposing a close relationship to it. There are a few negative impacts for certain variables in the inclusive curtor model but nothing too significant. Thus we may find that there is slight noise added for the variables that seemingly don't have a relationship with the x-ray data but overall it is worth adding the data for an accurate inverse model.

5 CONCLUSIONS AND FUTURE WORK

Across all models, we were able to determine a trend possibly unique to random forest regression models. The scaling of error seems to be affected by the range of answers the model can give. The trend is most notable when the model's prediction is poor; a rise in error as the test values approach extremes. The inverse power model revealed close ties of the power profile to ate0, lh_power, and ip_scale. The inverse curtor model revealed close ties of the current profile to ate0, elecfd, and ip_scale. Additionally, the inclusion of x-ray data in the inverse models revealed a relationship between the x-ray data and the R0_eqdsk variable.

This work sets the baseline for the creation and comparison of deep learning model for this inverse problem such as gaussian approximation and multilayer perceptron. The findings will be compared to others to see what trends are unique to the model at hand and what is unique to the data set we are observing.

ACKNOWLEDGMENTS

For the unfiltered data set we thank the fusion team at Berkeley and, for guidance and his expertise, I thank Wes E. Bethel.

REFERENCES

- [1] F. Maleki, K. Ovens, K. Najafian, B. Forghani, C. Md, and R. Forghani. Overview of machine learning part 1. *Neuroimaging Clinics of North America*, 30:e17–e32, 11 2020. doi: 10.1016/j.nic.2020.08.007
- [2] D. Moreau and I. Voitsekhovitch. Plasma control issues for an advanced steady state tokamak reactor. *Nuclear Fusion*, 36:685, 1999. doi: 10.1088/0029-5515/39/5/308
- [3] J. Moreno-Torres, J. A. Sáez, and F. Herrera. Study on the impact of partition-induced dataset shift on -fold cross-validation. *IEEE transactions on neural networks and learning systems*, 23:1304–12, 08 2012. doi: 10.1109/TNNLS.2012.2199516
- [4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [5] G. Wallace, Z. Bai, R. Sadre, T. Perciano, N. Bertelli, S. Shiraiwa, E. Bethel, and J. Wright. Towards fast and accurate predictions of radio frequency power deposition and current profile via data-driven modelling: applications to lower hybrid current drive. *Journal of Plasma Physics*, 88(4):895880401, 2022. doi: 10.1017/S0022377822000708
- [6] G. M. Wallace. Lower hybrid current drive physics overview, May 2023.